МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»

Кафедра інженерії програмного забезпечення

КУРСОВА РОБОТА

(ПОЯСНЮВАЛЬНА ЗАПИСКА)

з дисципліни: «Об’єктно-орієнтоване програмування»

на тему:

«РОЗРОБКА ГРИ «GEOMETRY DASH» »

студента І курсу групи ІПЗ-23-2
спеціальності 121 «Інженерія
програмного забезпечення»
Кузнєцової Ольги Олександрівни
(прізвище, ім’я та по-батькові)

Керівник: старший викладач кафедри
ІПЗ Олег Власенко

Дата захисту: " ___ " _____ 2025р.
Національна шкала _____
Кількість балів: _____
Оцінка: ECTS _____

Члени комісії

_____ Тетяна ВАКАЛЮК
(підпис) (прізвище та ініціали)

_____ Олег ВЛАСЕНКО
(підпис) (прізвище та ініціали)

_____ Олексій ЧИЖМОТРЯ
(підпис) (прізвище та ініціали)

Житомир – 2025

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
Факультет інформаційно-комп'ютерних технологій
Кафедра інженерії програмного забезпечення
Освітній рівень: бакалавр
Спеціальність 121 «Інженерія програмного забезпечення»

«ЗАТВЕРДЖУЮ»
Завідувач кафедри ІПЗ
_____ Тетяна ВАКАЛЮК
«____»_____2025 р.

ЗАВДАННЯ
НА КУРСОВУ РОБОТУ СТУДЕНТУ
Кузнєцовій Ользі Олександрівні

1. Тема роботи: Розробка гри "Geometry Dash",
   керівник роботи: ст. викладач кафедри ІПЗ Власенко Олег
2. Строк подання студентом: «19» травня 2024 р.
3. Вихідні дані до роботи: Розробити програму імітації гри "Geometry Dash".
4. Зміст розрахунково-пояснювальної записки (перелік питань, які підлягають розробці)
   1. Постановка завдання
   2. Аналіз аналогічних розробок
   3. Алгоритми роботи програми
   4. Опис роботи програми
   5. Програмне дослідження

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)
   1.Презентація до КР
   2.Посилання на репозиторій: _____

6. Консультанти розділів роботи

Розділ	Прізвище, ініціали та посади консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання "04" березня 2024 р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів курсового проєктування	Строк виконання етапів роботи	Примітки
1	Постановка задачі	05.03.2024	виконано
2	Пошук, огляд та аналіз аналогічних розробок	11.03.2024	виконано
3	Формулювання технічного завдання	13.03.2024	виконано
4	Опрацювання літературних джерел	20.03.2024	виконано
5	Проектування структури	01.04.2024	виконано
6	Написання програмного коду	12.04.2024	виконано
7	Відлагодження	31.05.2024	виконано
8	Написання пояснювальної записки	01.06.2024	виконано
9	Захист	09.11.2025	виконано

Студент _____ Ольга КУЗНЄЦОВА
(підпис) (прізвище та ініціали)

Керівник роботи _____ Олег ВЛАСЕНКО
(підпис) (прізвище та ініціали)

РЕФЕРАТ

Курсова робота присвячена розробці гри "Geometry Dash". Пояснювальна записка до курсової роботи на тему розробка гри "Geometry Dash" складається з: вступу, основної частини (трьох розділів), висновків, списку використаної літератури та додатків.

Текстова частина вкладена на 25 сторінках тексту. У вступі розглядаються актуальність теми, мета та завдання дослідження, а також коротко описується структура роботи. Основна частина складається з трьох розділів. У першому розділі аналізуються існуючі аналоги та визначаються основні вимоги до гри. У другому розділі детально описуються процес розробки гри, включаючи вибір технологій та інструментів, таких як C# та WPF. Також розглядається архітектурний паттерн MVVM. Третій розділ присвячений опису та тестуванню гри. Висновки підсумовують основні результати роботи та оцінюють досягнення поставлених завдань.

Пояснювальна записка має 38 сторінок додатків, де наводяться коди програм. В записці присутні приклади форм, схем, діаграм, таблиць та інших гра допоміжних матеріалів. Список використаних джерел містить 10 найменувань і займає 1 сторінку. В роботі наведено 12 рисунків, що ілюструють ключові етапи розробки та інтерфейс гри. Загальний обсяг роботи складає 63 сторінок.

Ключові слова: гра, C#, WPF, MVVM, програмне забезпечення.

ЗМІСТ

ВСТУП……. 6
РОЗДІЛ 1. АНАЛІЗ ПРОБЛЕМАТИКИ, МЕТОДІВ ТА ЗАСОБІВ ВИРІШЕННЯ ЗАДАЧІ…… 7
1.1 Аналіз задачі, засобів та методів її вирішення…… 7
1.2 Аналіз існуючого програмного забезпечення за тематикою курсового проекту (роботи)……... 8
Висновки до першого розділу……... 11
РОЗДІЛ 2. ПРОЕКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ…… …… 12
2.1 Проектування загального алгоритму роботи програми…… 12
2.2 Розробка функціональних алгоритмів роботи програми…….. 13
2.3 Розробка програмного забезпечення…….. 15
Висновки до другого розділу…… 16
РОЗДІЛ 3. ОПИС РОБОТИ З ПРОГРАМНИМ ДОДАТКОМ ТА ЙОГО ТЕСТУВАННЯ…….. 18
3.1 Опис роботи з програмним додатком (Опис інтерфейсу) ……... 18
3.2 Тестування роботи програмного забезпечення……. 22
Висновки до третього розділу……... 22
ВИСНОВКИ…….. 24
СПИСОК ЛІТЕРАТУРИ…… 25
ДОДАТКИ…… 26

ВСТУП

У сучасному культурному та технологічному просторі відеоігри посідають ключове місце, ставши невід'ємною складовою життя мільйонів людей у всьому світі. Завдяки стрімкому розвитку технологій, ігрова індустрія трансформувалася з простої розваги в потужний багатофункціональний інструмент. Ігри сьогодні активно впливають на ключові соціальні аспекти: від освіти, психології та соціальних комунікацій до світової економіки. Вони пропонують не лише можливість поринути у захоплюючі віртуальні світи, але й ефективно стимулюють розвиток креативності, відточують стратегічне мислення та зміцнюють навички командної роботи. У глобалізованому цифровому середовищі ігри набули безпрецедентної актуальності, перетворившись із засобу відпочинку на важливий канал для самовираження та соціальної взаємодії.

Ця курсова робота присвячена розробці гри "Geometry Dash".

У рамках курсової роботи предметом дослідження є жанр ритмічних платформерів його ключові механіки, що поєднують точність рухів та синхронізацію з аудіорядом, а також особливості візуального стилю, характерного для "Geometry Dash".

Об'єктом дослідження виступає процес створення програмного забезпечення гри, включаючи реалізацію фізичної моделі руху персонажа (стрибки та гравітація), систему обробки колізій із різними типами перешкод, а також інтеграцію циклічного ігрового процесу на базі технології WPF (Windows Presentation Foundation) та мови C#.

Особливу увагу в дослідженні приділяється механіці гри, інтерактивності ігрового процесу, а також оптимізації коду для досягнення максимальної продуктивності ігрового платформера.

Метою курсової роботи є набуття практичного досвіду у створенні гри в жанрі ритмічного платформера з використанням технології WPF та мови C#.

РОЗДІЛ 1.АНАЛІЗ ПРОБЛЕМАТИКИ, МЕТОДІВ ТА ЗАСОБІВ ВИРІШЕННЯ ЗАДАЧІ

1.1Аналіз задачі, засобів та методів її вирішення

У процесі розробки динамічного 2D платформера-раннера у стилі «Geometry Dash» необхідно провести аналіз задачі, засобів та методів її вирішення, які визначають якість та функціональність кінцевого продукту.

Задачі даної гри :

Створення цікавого та точного керування стрибками, забезпечення чіткої інтеграції між швидкістю гри та музичним ритмом, а також розробка системи нерухомих перешкод. Забезпечення інтуїтивно зрозумілого інтерфейсу користувача, зручного меню та екранів результатів. Підбір або створення аудіодоріжок та звукових ефектів, які синхронізуються з ігровим процесом, посилюючи відчуття ритму та динаміки.

Забезпечення візуальної привабливості гри через чіткі геометричні форми об'єктів, оригінальний дизайн та динамічну анімацію гравця, заохочення до багаторазового проходження гри шляхом інтеграції кількох унікальних, попередньо розроблених рівнів.

Методи:

Для досягнення цих цілей буде використано мову програмування C#, платформу розробки: WPF (Windows Presentation Foundation). Ця технологія дозволяє ефективно працювати з графічними елементами (XAML) та прив'язкою даних, що є ідеальним для розробки інтерфейсів та 2D ігор, архітектурний паттерн MVVM. Розробка через ітерації: послідовні внесені зміни, які відповідатимуть потребам гравців допоможуть грі вийти на новий рівень.

1.2Аналіз існуючого програмного забезпечення за тематикою курсового проекту (роботи).

Аналіз існуючого програмного забезпечення у жанрі **динамічних платформерів-раннерів**, або так званих "ритм-платформерів", є одним з основних етапів у процесі розробки. На сучасному ринку існує безліч програмних продуктів, які відрізняються за складністю, візуальним стилем та ступенем інтеграції ритмічних елементів. З метою ефективного визначення ключових особливостей та унікальних переваг власного проєкту, важливо провести ретельний аналіз конкурентів, оцінити їхні сильні та слабкі сторони, а також визначити відповідність їхнього підходу до розробки на платформі **WPF C#** (де акцент робиться на чіткій, геометричній графіці).

В якості ключових аналогів для нашого проєкту, які відображають основні механіки ритму та точного управління, були обрані наступні ігри:

1. The Impossible Game (2009):

Автор: Grip Games

Джерело: https://gamedev.dou.ua/articles/helldivers-2-review/

Опис джерела:https://store.steampowered.com/app/251630/The_Impossible_Game

The Impossible Game – гра-попередник Geometry Dash. Базується на простій геометричній графіці та мінімалістичному управлінні (одна кнопка). Рівні фіксовані та вимагають ідеального запам'ятовування.

Переваги:

- Простота механіки(Дозволяє зосередитися на таймінгу).
- Чітка графіка(Геометричні форми легко відтворюються).

Недоліки:

- Відсутність варіативності (Мало рівнів і можливостей кастомізації).
- Сувора складність(Часто є джерелом фрустрації).

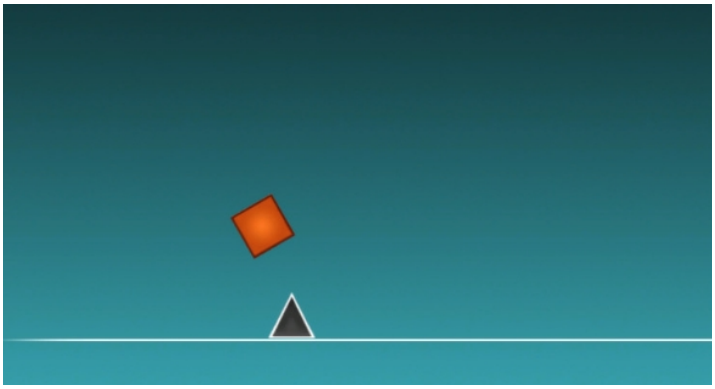

Рис.1.1 – The Impossible Game

2. Electronic Super Joy (2013)

Автор: Michael Todd Games

Джерело: https://store.steampowered.com/app/244870/Electronic_Super_Joy/

Electronic Super Joy — це надзвичайно складний платформер, дія якого розгортається у світі запальної електронної музики. Біжіть, стрибайте, трощіть і літайте, проходячи понад 45 дивних і різноманітних рівнів із низькою гравітацією, обертанням світу, гігантськими монстрами та роями ракет.

Переваги:

- Візуальний стиль: Неонові кольори та динамічні ефекти добре поєднуються з графічними можливостями.
- Енергійний саундтрек та інтерактивність.

Недоліки:

- Складність: Може бути занадто хаотичним для гравця, який шукає чистий ритм.

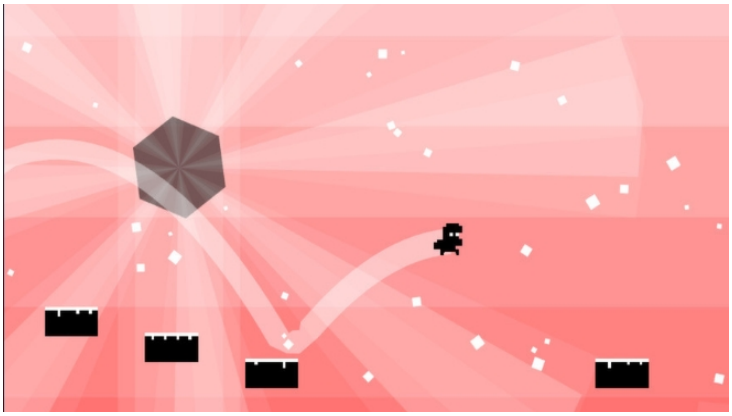

Рис.1.2 – Electronic Super Joy

3. BIT.TRIP Runner (2010)

Автор: Gaijin Games, Choice Provisions

Джерело: https://store.steampowered.com/app/63710/BITTRIP_RUNNER/

Опис джерела:

BIT.TRIP RUNNER — це більше, ніж просто гра; це шедевр у стилі ретро з піксельною графікою, який віддає данину поваги класичним іграм, одночасно пропонуючи сучасну та захоплюючу пригоду. Простота управління робить гру доступною для всіх, але для її освоєння потрібні навички, відчуття часу та глибокий зв'язок з музикою. Ця унікальна суміш складного ігрового процесу та захоплюючої естетики виділяє BIT.TRIP RUNNER серед інших ігор, роблячи її обов'язковою для любителів жанру та новачків.

Переваги:

- Чиста інтеграція музики та геймплею: Кожна дія є частиною ритму.
- Прогресивна складність та візуальна чіткість

Недоліки:

- Жорсткі часові рамки: Гра майже не допускає помилок у таймінгу, що може бути надто вимогливим.
- Занадто багато деталей, важко сконцентруватися на перешкодах.

Рис.1.3 – BIT.TRIP RUNNER

Загальний аналіз програмного забезпечення підтверджує різноманітність варіантів схожих до гри Geometry Dash. Кожна гра має свої переваги та недоліки, і вибір залежить від конкретних потреб та вподобань гравців Розробка власної гри дозволить ефективно вирішити конкретні завдання, забезпечуючи високий рівень користувацької зручності.

Висновки до першого розділу

Проведений загальний аналіз ігрового програмного забезпечення підтвердив різноманітність існуючих рішень у жанрі динамічних платформерів, схожих на Geometry Dash.

Кожен із проаналізованих аналогів має свої сильні та слабкі сторони, і вибір кінцевої реалізації механік залежить від конкретних потреб проєкту. Розробка власної гри на платформі WPF C# дозволяє цілеспрямовано використати архітектуру MVVM для ефективного вирішення конкретних технічних завдань і забезпечення високого рівня модульності та користувацької зручності.

У цьому розділі я провела детальний аналіз задачі створення гри та обґрунтували вибір засобів і методів її реалізації (мова програмування C#, платформа WPF, паттерн MVVM).

Було детально розглянуто ключові аналоги ігор у жанрі ритм-платформерів.

Я дослідила наявні ігри, щоб зрозуміти, які елементи геймплею та механіки (зокрема, точність таймінгу та синхронізація з музикою) є найпопулярнішими серед гравців.

РОЗДІЛ 2. ПРОЕКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Проектування загального алгоритму роботи програми

Перейдемо до проектування програмного додатку. Метою цього етапу розроблення загальних алгоритмів роботи, які дозволяють оцінити роботу з точки зору користувача.

Функції які доступні для користувача :

1. Увійти в гру.
2. Обрати рівень гри.
3. Використання паузи під час гри.
4. Перезапуск гри після проходження рівня.
5. Доступні ще два рівні гри.
6. Завершення гри.

Ці функції є основними можливостями власної гри в жанрі «Geometry Dash». Загальні функції для користувача відображені на схемі Рис.2.1.

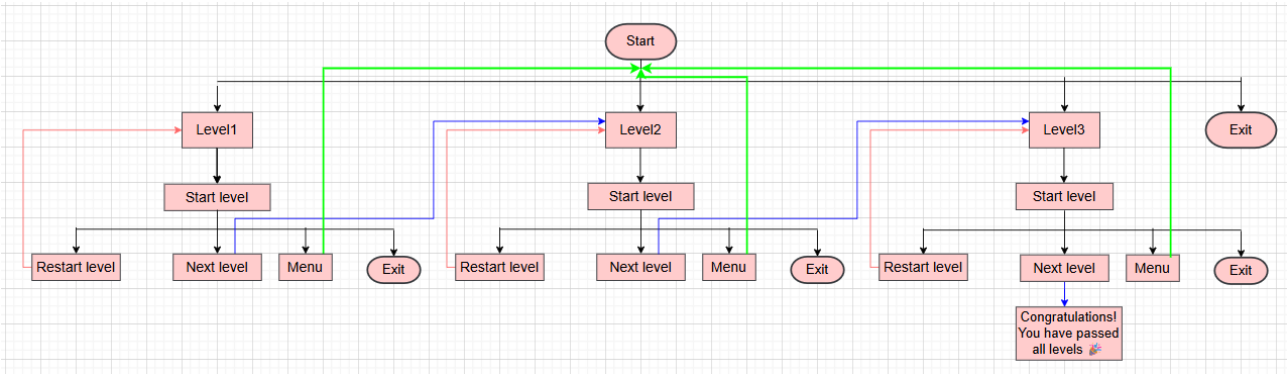

Рис.2.1 – Схема роботи програми

2.2 Розробка функціональних алгоритмів роботи програми

При створенні гри особлива увага приділялася уникненню зайвих елементів в інтерфейсі, які могли б відволікати користувача від швидкого ігрового процесу. Завдяки цьому інтерфейс вийшов мінімалістичним та інтуїтивно зрозумілим.

При запуску рівня ініціалізуються всі ігрові об'єкти (гравець, колекція перешкод) та запускається єдиний ігровий цикл, який керується ViewModel.

Ключові алгоритми гри: **Рух та фізика** - в ігровому циклі постійно оновлюється вертикальна позиція гравця під дією сили гравітації та вектора стрибка. Усі перешкоди рухаються по горизонталі назустріч гравцю з фіксованою швидкістю. **Управління**: гравець взаємодіє з грою однією дією — стрибком. Ця дія викликає відповідну команду у ViewModel. **Визначення результату** (Програш): Якщо гравець стикається з небезпечною перешкодою (шип, блок) або торкається платформи небезпечним способом (збоку чи знизу), фіксується зіткнення, і гра завершується (програш). **Визначення результату** (Перемога): Якщо гравець успішно проходить усі перешкоди рівня та досягає фінальної контрольної точки, рівень вважається пройденим. (Рис.2.1)

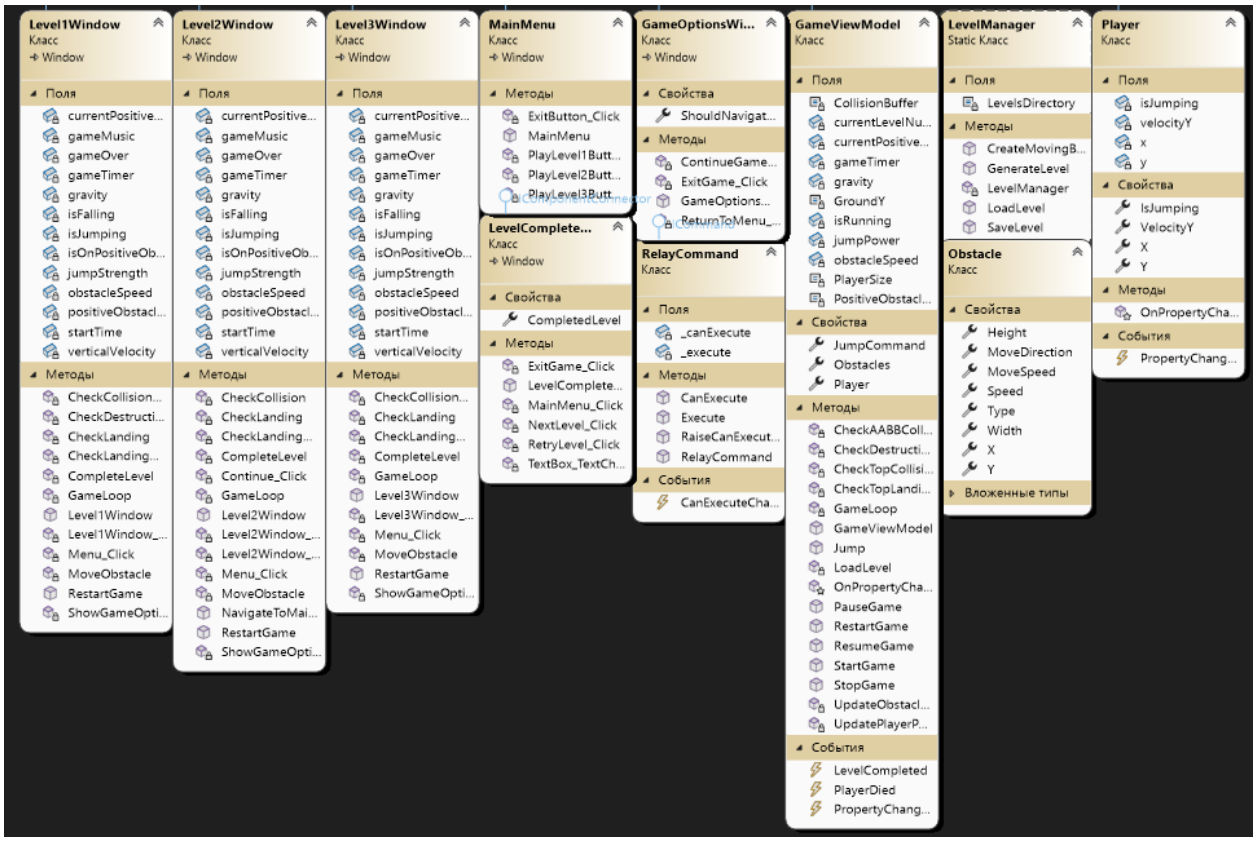

Рис.2.2 – Діаграма класів

Діаграма класів, що представлена на зображенні (Рис. 2.2), детально описує структуру та взаємозв'язки між ключовими компонентами проєкту Geometry Dash. Ця чітка структура, побудована з використанням фреймворку WPF та архітектурного паттерна MVVM, слугує надійною основою для подальшого розвитку, масштабування та модифікації системи.

У цій WPF-грі, побудованій за паттерном MVVM, клас GameViewModel є головним класом, що відповідає за управління всією ігровою логікою та забезпечення взаємодії між моделями (Player, Obstacle) та представленням (Views).

Модель даних: GameViewModel містить посилання на ключові ігрові об'єкти (Моделі), такі як Player (кубик-гравець) та ObservableCollection<Obstacle> (колекція перешкод для поточного рівня). Ігровий цикл та фізика: Клас реалізує управління ігровим таймером (gameTimer) для постійного оновлення ігрового стану (метод GameLoop). Сюди входить логіка гравітації, вектор стрибка (jumpPower), горизонтальний рух перешкод та складні перевірки зіткнень (CheckCollision та CheckLanding). Управління станом: GameViewModel забезпечує управління станом гри (запуск, пауза, перезапуск — StartGame, StopGame, RestartGame) та повідомляє View про критичні події через власні події (PlayerDied, LevelCompleted).

Ключовий функціонал класу включає методи для:

Обробки вводу: Керування стрибком через команду, забезпечуючи чисту прив'язку даних без прямої взаємодії з клавіатурою у View.

Управління рухом: Оновлення позицій гравця та перешкод (UpdatePlayerPosition, UpdateObstaclePositions).

2.3 Розробка програмного забезпечення

Після наведення основних алгоритмів роботи програми можна перейти до опису основних реалізованих методів.

Метод	**Опис роботи**
LoadLevel()	Очищує колекцію Obstacles та завантажує новий набір перешкод для поточного рівня (currentLevelNumber), використовуючи LevelManager.GenerateLevel().
CheckTopLandingCollision()	Перевіряє, чи знаходиться гравець у зоні безпечного приземлення (зверху) на позитивну перешкоду. Якщо так, повертає об'єкт перешкоди.
CheckTopCollisionOnly()	Використовується в логіці смертельного зіткнення, щоб визначити, чи було зіткнення з платформою небезпечним (не зверху).
CheckDestructiveCollisions()	Проходить по всіх перешкодах та перевіряє смертельне зіткнення за допомогою CheckAABBCollision(). У разі смерті викликає StopGame() та подію PlayerDied.
CheckAABBCollision()	Базовий метод для визначення перетину прямокутних меж гравця та перешкоди.
StartGame(), StopGame(), PauseGame(), ResumeGame()	Керування життєвим циклом гри: запуск/зупинка таймера та зміна прапора isRunning.
RestartGame()	Скидає стан гравця (позиція, швидкість, стан стрибка) та повторно викликає LoadLevel() та StartGame().
OnPropertyChanged()	Стандартний метод реалізації інтерфейсу INotifyPropertyChanged, що повідомляє View про зміну значень властивостей.

LoadLevel(string levelName)	-	Зчитує дані рівня з файлу JSON та десеріалізує їх у список об'єктів Obstacle.
GenerateLevel(int levelNumber)	-	Містить хардкод структури та початкових параметрів перешкод для рівнів 1 та 2 (визначає, які об'єкти створюються).
CreateMovingBlock()	-	Допоміжний метод для створення об'єкта Obstacle з певним типом та параметрами руху.
LevelXWindow_KeyDown()	Level1Window	Обробник натискання клавіш. Захоплює натискання Space і викликає команду стрибка. Також обробляє Shift для відкриття меню опцій.
Menu_Click()	Level1Window	Обробник натискання кнопки "Menu". Викликає метод ShowGameOptions().
ShowGameOptions()	Level1Window	Ставить гру та музику на паузу, відкриває вікно GameOptionsWindow як діалог, а після закриття — відновлює гру або переходить до головного меню.
CompleteLevel()	Level1Window	Зупиняє гру, створює та показує вікно LevelCompleteWindow, передаючи назву пройденого рівня, і закриває поточне вікно.

ReturnToMenu_Click()	GameOptionsWindow	Обробник кнопки "Return to Menu". Створює та показує MainMenu, закриваючи поточне вікно гри.
Конструктор MainMenu()	MainMenu	Ініціалізує вікно. Оскільки це головне меню, він слугує точкою, з якої починається навігація в грі.
PlayLevel1Button_Click()	MainMenu	Обробник кнопки "Play Level 1". Створює новий екземпляр Level1Window, викликає метод Show() для відображення вікна рівня та закриває поточне вікно меню (this.Close()). Аналогічні методи є для Level 2 та Level 3.
ExitButton_Click()	MainMenu	Обробник кнопки "Exit". Завершує роботу всього застосунку, викликаючи Application.Current.Shutdown().
TextBox_TextChanged()	LevelCompleteWindow	Цей метод присутній у вашому XAML та C# для LevelCompleteWindow, але в тілі коду він порожній і не використовується. Його варто згадати як технічно присутній обробник події, який не містить логіки.
Конструктор App()	App	Конструктор головного класу застосунку. Запускає ініціалізацію

		WPF-додатка.
InitializeComponent()	Усі View-класи (Window, App)	Обов'язковий метод WPF. Генерується автоматично та викликається в конструкторі кожного View. Відповідає за завантаження та ініціалізацію всіх елементів інтерфейсу, описаних у XAML-файлі.

РОЗДІЛ 3. ОПИС РОБОТИ З ПРОГРАМНИМ ДОДАТКОМ ТА ЙОГО ТЕСТУВАННЯ

3.1 Опис роботи з програмним додатком (Опис інтерфейсу)

Тепер опишемо інтерфейс, тим самим зробимо інструкцію для користувача.

Після запуску гри користувач побачить стартове меню. На ньому є кнопки: Start, Play Level 1, Play Level 2, Play Level 3, Exit. (Рис.3.1)

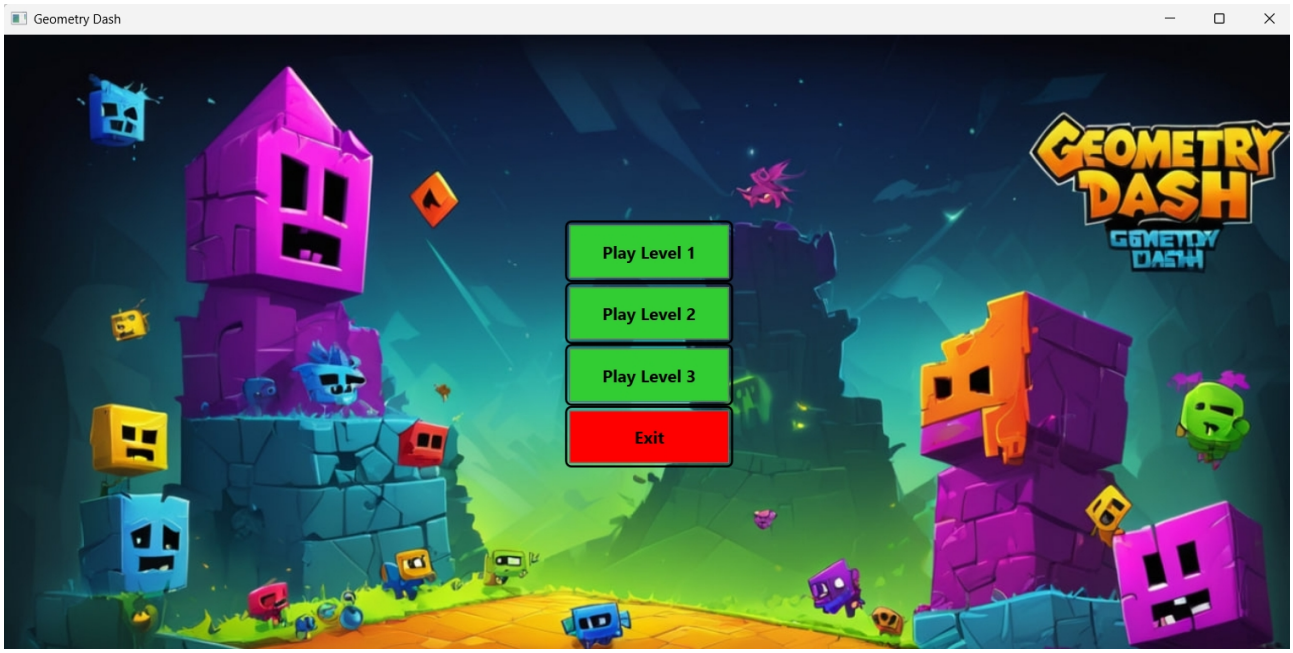

Рис.3.1 – Стартове меню

Після того як користувач натисне Play Level 1, почнеться гра.(Рис.3.2)

Рис.3.2 – Запущений level1

Натиснувши на кнопку паузи гра буде призупинена (рис.3.3) і будуть показані функції які можливо зробити (продовжити гру, вихід в меню, вихід з гри).

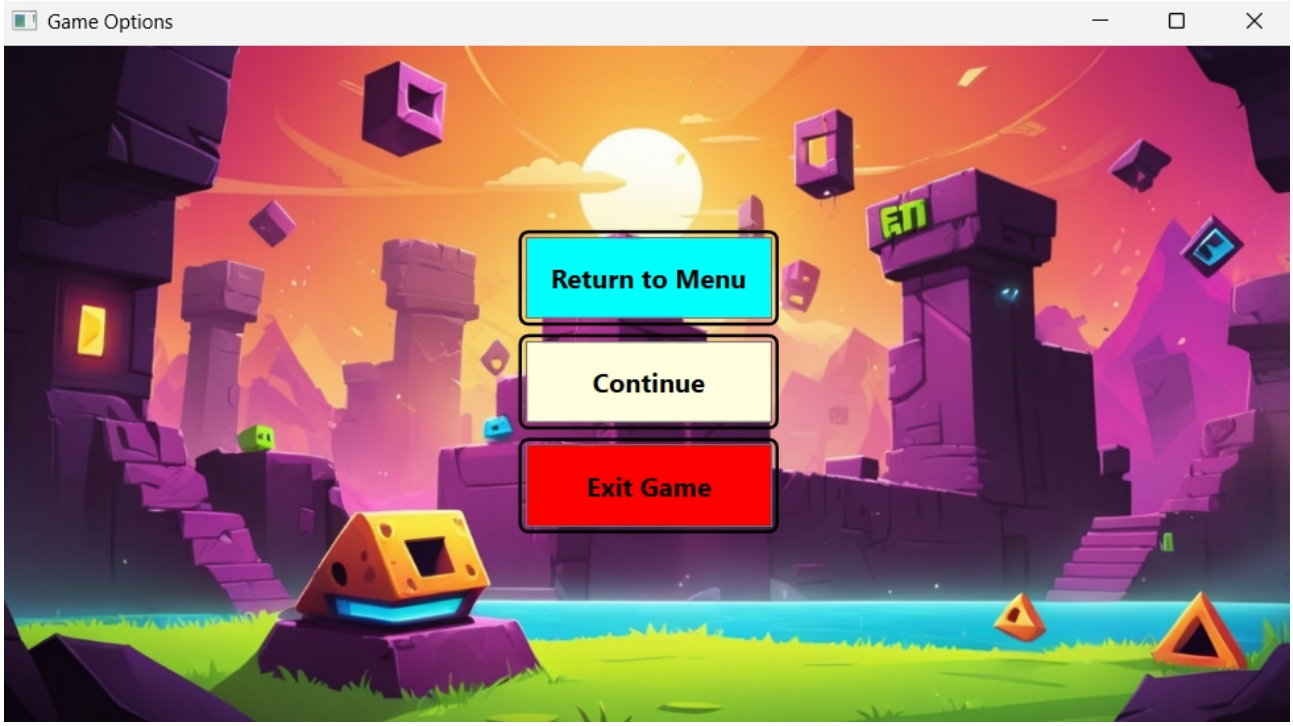

Рис.3.3 – Гра на паузі

Після проходження рівня нас вітають і відкривається вікно де буде показано функції які можливо зробити (переграти рівень, увімкнути наступний рівень, вихід у меню, вихід з гри). (Рис.3.4).

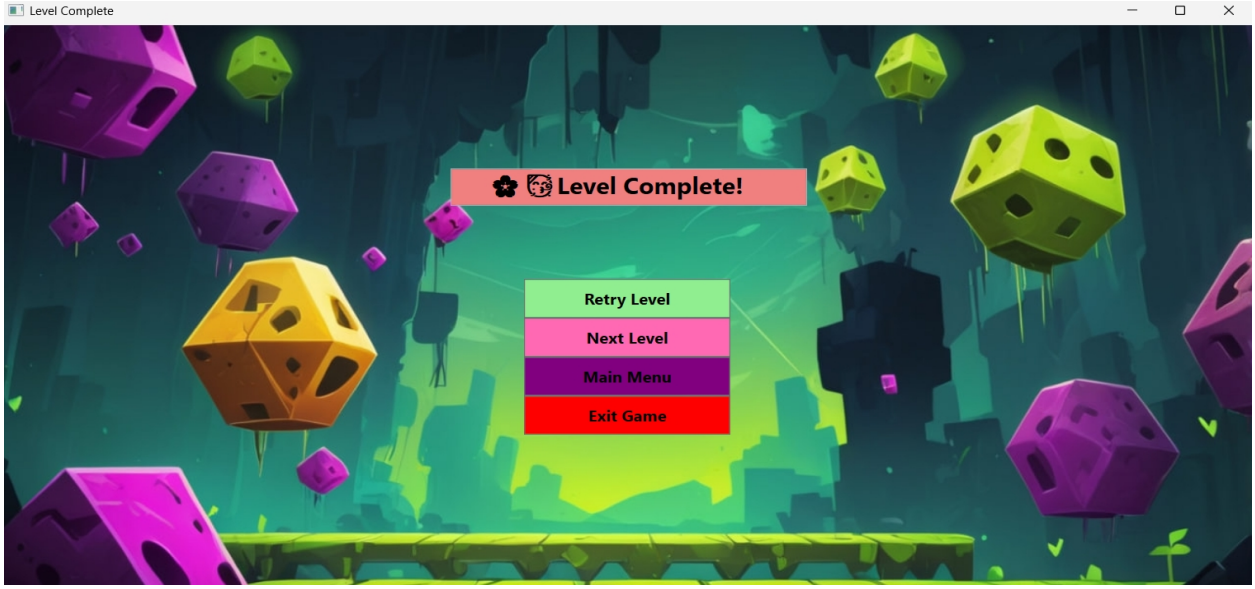

Рис.3.4 – меню після проходження рівня

В разі поразки (зіткнення із перешкодами рівень автоматично починається спочатку). При натисненні кнопки Next Level після проходження рівня відкривається вікно другого рівня і починається гра. (Рис.3.5)

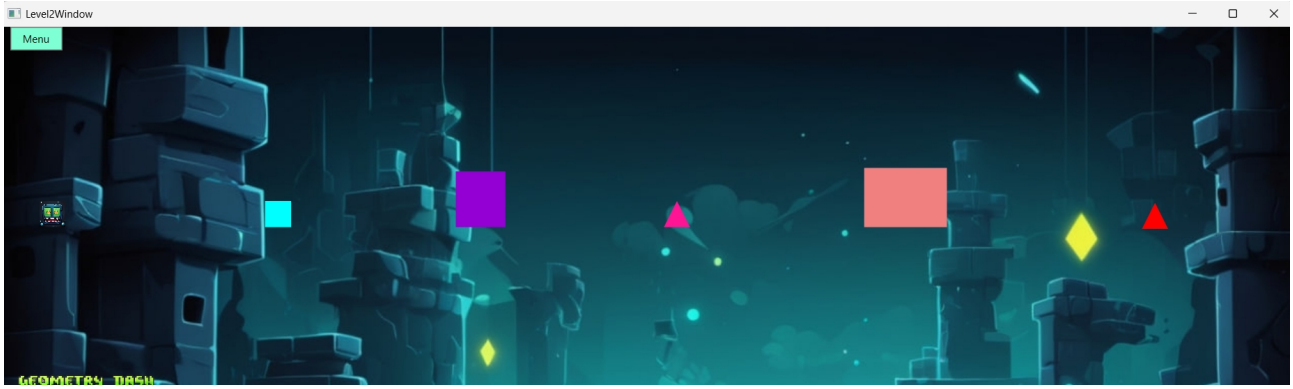

Рис.3.5 – level2

Після проходження рівня нас знову вітають і відкривається вікно де буде показано функції які можливо зробити (переграти рівень, увімкнути наступний рівень, вихід у меню, вихід з гри). (Рис.3.4). При натисненні кнопки Next Level після проходження рівня відкривається вікно другого рівня і починається гра. (Рис.3.6)

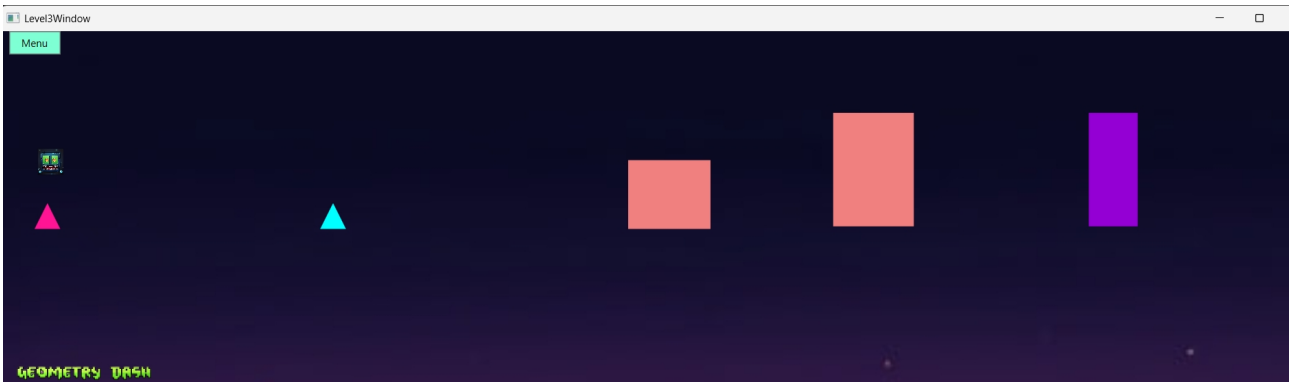

Рис.3.6 – level3

Після проходжень всіх рівнів(3-ого) при натисканні на кнопку Next level нас вітають з проходженням всіх рівнів. (Рис.3.7)

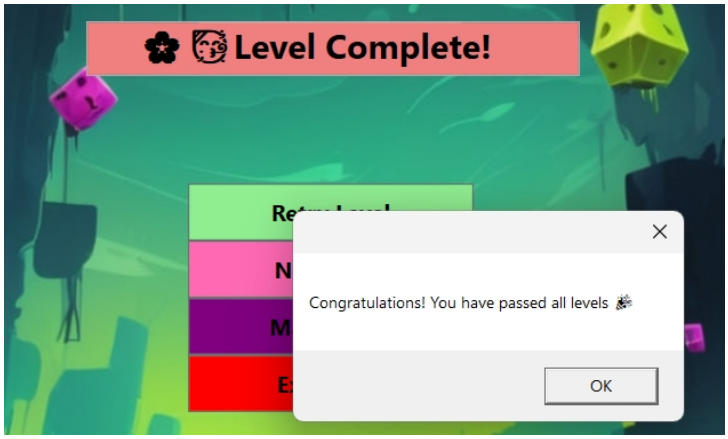

Рис.3.7 – level3

Ця гра є нескінченною, а саме рівні можна проходити безліч разів. Кожен рівень відрізняється швидкістю руху перешкод, різна музика, різне розташування перешкод.

3.2 Тестування роботи програмного забезпечення

У цьому розділі я провела тестування ключових функціональних аспектів динамічного 2D платформера Geometry Dash, розробленого на WPF C#, для виявлення можливих вірогідних помилок, які можуть вплинути на ігровий досвід (UX).

Основна увага в тестуванні приділяється точності механіки зіткнень та синхронізації ігрового циклу з візуальним відображенням. Тестування проводилося шляхом багаторазового проходження рівнів та фіксації критичних моментів взаємодії гравця з перешкодами.

Під час тестування було виявлено та зафіксовано недолік, що стосується точності визначення зіткнень:

Іноді гравець зазнає програшу (зіткнення зараховується), коли візуально кубик ще не повністю торкнувся краю небезпечної перешкоди, або ж, навпаки, момент приземлення на платформу вимагає занадто високої точності. Це пов'язано з використанням AABB-зіткнень (Axis-Aligned Bounding Box) та частотою оновлення ігрового циклу, що може призводити до незначного проникнення об'єктів або відчуття "запізнення" в долі секунди.

Цей недолік має бути виправлений у майбутньому шляхом впровадження більш точного алгоритму визначення зіткнень.

Висновки до третього розділу

У третьому розділі було детально розглянуто та протестовано основний функціонал гри Geometry Dash, що дало змогу виявити потенційні помилки та підтвердити працездатність системи. Нашою метою було забезпечення коректної роботи всіх ігрових механік та загальне задоволення потреб гравців.

Створений динамічний платформер-раннер пропонує користувачам такі ключові можливості:

Основний ігровий процес: Точне управління кубиком (гравцем) за допомогою єдиної дії — стрибка, у повній синхронізації з музичним ритмом рівня.

Функціонал: Наявність чіткого меню паузи з опціями продовження та виходу.

Користувачі можуть управляти гравцем, уникаючи зіткнень із небезпечними перешкодами та прагнучи до повного проходження рівня.

Під час тестування було виявлено та зафіксовано один істотний недолік, пов'язаний з точністю визначення колізій: іноді виникає передчасне спрацювання зіткнення гравця з перешкодою. Також були помічені незначні проблеми з часом відгуку інтерфейсу, які хоч і не критично впливають на геймплей, але вимагатимуть оптимізації у майбутніх версіях гри.

Загалом, гра Geometry Dash демонструє задовільний рівень функціональності та стабільності, особливо враховуючи її реалізацію на фреймворку WPF C# із застосуванням архітектури MVVM. Проте, для подальшого вдосконалення гри та підвищення зручності користування рекомендується пріоритетне виправлення виявлених помилок та проведення додаткового тестування.

ВИСНОВКИ

Під час виконання даної курсової роботи було ретельно проаналізовано основні аспекти розробки динамічного 2D платформера-раннера у стилі Geometry Dash. Аналізуючи існуючі продукти, ми дійшли висновку, що високий попит існує на ігри, які поєднують точне управління, музичний ритм та простоту класичних аркад. Це підтвердило доцільність розробки власного проєкту, який поєднує мінімалістичний ретро-стиль з сучасними технологіями та архітектурними підходами.

Реалізація проєкту на платформі WPF із використанням мови програмування C# виявилася вимогливою і водночас захопливою. Навіть при наочній простоті візуальної теми, розробка ігрового циклу виявила низку технічних складнощів, особливо у сфері точності фізики та визначення зіткнень (AABB). Застосування архітектурного паттерна MVVM дозволило ефективно подолати ці складнощі, забезпечивши чітке відокремлення ігрової логіки (GameViewModel) від графічного інтерфейсу (Views).

Основний фокус розробки було спрямовано на створення інтуїтивно зрозумілого інтерфейсу та захопливого ігрового процесу, оскільки ці фактори мають вирішальне значення для майбутніх гравців. Процес розробки включав не лише написання програмного коду та структурування рівнів за допомогою LevelManager, але й ручне тестування гри на різних етапах.

Тестування дозволило виявити, що існуючі дефекти та недоліки, зокрема неточність спрацювання колізій, наразі не перешкоджають функціональному користуванню програмою. Проте, ці аспекти потребують уваги та підготовлено план їх подальших вдосконалень.

Отже, результатом роботи стала гра Geometry Dash, реалізована на мові програмування C# з використанням фреймворку WPF та паттерну MVVM. Хоча цей проєкт не є абсолютно досконалим, він демонструє повну

функціональність та стабільність. Робота над ним буде продовжена, зосереджуючись на підвищенні якості продукту, усуненні виявлених дефектів та задоволенні потреб гравців у високоякісному ритм-платформері.

СПИСОК ЛІТЕРАТУРИ

1. Pro C# 7: With .NET and .NET Core / Andrew Troelsen, Philip Japikse. – Apress, 2017. – 1370 с.
2. WPF 4. Докладне керівництво, Адам Натан.
3. Технології програмування. Мова C# : навч. посібн/ В.В. Томашевський. – Житомир: ЖВІ НАУ, 2012. – 484 с.
4. Expert C# 5.0: with the .NET 4.5 Framework / Mohamed Elgendy. – CreateSpace Independent Publishing Platform, 2014. – 450 с.
5. Programming WPF / Charles Petzold.
6. C# 7.0 in a Nutshell: The Definitive Reference / Joseph Albahari, Ben Albahari. – O'Reilly Media, 2017. – 1088 с.
7. Pro WPF in C#: Windows Presentation Foundation in .NET / Adam Freeman.
8. Microsoft Visual C# Step by Step / John Sharp. – Microsoft Press, 2018. – 992 с.
9. Beginning C# 7 Programming with Visual Studio 2017 / Benjamin Perkins, Jacob Vibe Hammer, Jon D. Reid. – Wrox, 2018. – 800 с.
10. WPF and XAML Controls - Microsoft Docs / Microsoft.

ДОДАТКИ

Додаток А

Models(GameScreenModels)

LevelManager.cs:

```
using System;
using System.Collections.Generic;
using System.IO;
using GeometryDash.Models;
using Newtonsoft.Json;
using static GeometryDash.Models.Obstacle;

namespace GeometryDash.Managers
{
    public static class LevelManager
    {
        private const string LevelsDirectory = "Levels";

        static LevelManager()
        {
            if (!Directory.Exists(LevelsDirectory))
            {
                Directory.CreateDirectory(LevelsDirectory);
            }
        }

        public static void SaveLevel(string levelName, List<Obstacle> obstacles)
        {
            var levelData = JsonConvert.SerializeObject(obstacles);
            File.WriteAllText(Path.Combine(LevelsDirectory, $"{levelName}.json"),
levelData);
        }

        public static List<Obstacle> LoadLevel(string levelName)
        {
            var filePath = Path.Combine(LevelsDirectory, $"{levelName}.json");
            if (File.Exists(filePath))
            {
                var levelData = File.ReadAllText(filePath);
                return JsonConvert.DeserializeObject<List<Obstacle>>(levelData);
            }

            return new List<Obstacle>();
        }

        public static List<Obstacle> GenerateLevel(int levelNumber)
        {
            List<Obstacle> obstacles = new List<Obstacle>();

            switch (levelNumber)
            {
                case 1:
```

```
obstacles.Add(new Obstacle { X = 400, Y = 200, Width = 30, Height = 30, Speed = 10 });
                    obstacles.Add(new Obstacle { X = 600, Y = 200, Width = 30, Height
= 30, Speed = 8 });
                    break;
                case 2:
                    obstacles.Add(new Obstacle { X = 400, Y = 200, Width = 30, Height
= 30, Speed = 10 });
                    obstacles.Add(new Obstacle { X = 600, Y = 200, Width = 30, Height
= 30, Speed = 8 });
                    obstacles.Add(new Obstacle { X = 700, Y = 150, Width = 50, Height
= 50, Speed = 12 });
                    break;

                default:
                    throw new NotImplementedException("Level not implemented.");
            }

            return obstacles;
        }



        public static Obstacle CreateMovingBlock(double x, double y, double width,
double height, double moveDirection, double moveSpeed)
        {
            return new Obstacle { X = x, Y = y, Width = width, Height = height, Type =
ObstacleType.MovingBlock, MoveDirection = moveDirection, MoveSpeed = moveSpeed };
        }

    }
}
```

Obstacle.cs:

```
using System.Windows.Shapes;

namespace GeometryDash.Models
{
    public class Obstacle
    {
        public double X { get; set; }
        public double Y { get; set; }
        public double Width { get; set; }
        public double Height { get; set; }
        public double Speed { get; set; }
        public ObstacleType Type { get; set; }

        public double MoveDirection { get; set; }
        public double MoveSpeed { get; set; }

        public enum ObstacleType
        {
            Simple,
            MovingBlock,
            Positive
        }

    }
}
```

Player.cs:

```
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace GeometryDash.Models
{
    public class Player : INotifyPropertyChanged
    {
        private double x;
        private double y;
        private bool isJumping;
        private double velocityY;

        public double X
        {
            get => x;
            set { x = value; OnPropertyChanged(); }
        }
        public double Y
        {
            get => y;
            set { y = value; OnPropertyChanged(); }
        }
        public bool IsJumping
        {
            get => isJumping;
            set { isJumping = value; OnPropertyChanged(); }
        }
        public double VelocityY
        {
            get => velocityY;
            set { velocityY = value; OnPropertyChanged(); }
        }

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged([CallerMemberName] string propertyName =
null)
        {
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
        }
    }
}
```

Додаток Б

ViewModels

GameViewModel.cs:

```
using GeometryDash.Managers;
using GeometryDash.Models;
using System;
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Windows.Input;
using System.Windows.Threading;
using static GeometryDash.Models.Obstacle;
```

```
public class RelayCommand : ICommand
{
    private readonly Action<object> _execute;
    private readonly Func<object, bool> _canExecute;

    public RelayCommand(Action<object> execute, Func<object, bool> canExecute = null)
    {
        _execute = execute;
        _canExecute = canExecute;
    }
    public bool CanExecute(object parameter) => _canExecute == null ||
_canExecute(parameter);
    public void Execute(object parameter) => _execute(parameter);
    public event EventHandler CanExecuteChanged;
    public void RaiseCanExecuteChanged() => CanExecuteChanged?.Invoke(this,
EventArgs.Empty);
}

namespace GeometryDash.ViewModels
{
    public class GameViewModel : INotifyPropertyChanged
    {
        private const double PlayerSize = 30;
        private const double GroundY = 200;
        private const double PositiveObstacleSpeedMultiplier = 0.3;
        private const double CollisionBuffer = 10;

        private DispatcherTimer gameTimer;
        private double gravity = 1;
        private double jumpPower = 20;
        private double obstacleSpeed = 7;
        private int currentLevelNumber;
        private bool isRunning;
        private Obstacle currentPositiveObstacle = null;

        public Player Player { get; set; }
        public ObservableCollection<Obstacle> Obstacles { get; set; }
        public ICommand JumpCommand { get; }

        public event Action LevelCompleted;
        public event Action PlayerDied;

        public GameViewModel(int levelNumber)
        {
            currentLevelNumber = levelNumber;
            Player = new Player { X = 50, Y = GroundY, IsJumping = false, VelocityY =
0 };

            LoadLevel();

            gameTimer = new DispatcherTimer();
            gameTimer.Interval = TimeSpan.FromMilliseconds(15);
            gameTimer.Tick += GameLoop;
            StartGame();

            JumpCommand = new RelayCommand(Jump, (p) => !Player.IsJumping);
        }

        private void LoadLevel()
        {
            if (Obstacles == null)
            {
                Obstacles = new ObservableCollection<Obstacle>();
```

```
            }
            Obstacles.Clear();
            foreach (var item in LevelManager.GenerateLevel(currentLevelNumber))
            {
                Obstacles.Add(item);
            }
        }

        private void GameLoop(object sender, EventArgs e)
        {
            if (!isRunning) return;

            UpdatePlayerPosition();

            UpdateObstaclePositions();

            CheckDestructiveCollisions();

            if (Obstacles.Any())
            {
                var lastObstacle = Obstacles.Last();
                if (Player.X > lastObstacle.X + lastObstacle.Width * 2)
                {
                    StopGame();
                    LevelCompleted?.Invoke();
                }
            }
        }

        private void UpdatePlayerPosition()
        {
            if (Player.IsJumping)
            {
                Player.VelocityY += gravity;
                Player.Y += Player.VelocityY;
            }

            Obstacle landingObstacle = CheckTopLandingCollision();

            if (landingObstacle != null)
            {
                Player.Y = landingObstacle.Y - PlayerSize;
                Player.IsJumping = false;
                Player.VelocityY = 0;
                currentPositiveObstacle = landingObstacle;
            }
            else if (Player.Y >= GroundY)
            {
                Player.Y = GroundY;
                Player.IsJumping = false;
                Player.VelocityY = 0;
                currentPositiveObstacle = null;
            }

            if (currentPositiveObstacle != null)
            {
                if (Player.X > currentPositiveObstacle.X +
currentPositiveObstacle.Width - PlayerSize / 2)
                {
                    currentPositiveObstacle = null;
                    Player.IsJumping = true;
                }
                else
```

```
                {
                    Player.X -= (obstacleSpeed * PositiveObstacleSpeedMultiplier);
                }
            }
        }

        private void UpdateObstaclePositions()
        {
            foreach (var obstacle in Obstacles)
            {
                obstacle.X -= obstacleSpeed;

                if (obstacle.X < -obstacle.Width)
                {
                    obstacle.X = 1500;
                }
            }
        }

        private void CheckDestructiveCollisions()
        {
            foreach (var obstacle in Obstacles)
            {
                if (obstacle.Type == ObstacleType.Simple || obstacle.Type ==
ObstacleType.MovingBlock)
                {
                    if (CheckAABBCollision(Player, obstacle))
                    {
                        StopGame();
                        PlayerDied?.Invoke();
                        return;
                    }
                }
                else if (obstacle.Type == ObstacleType.Positive)
                {
                    if (CheckAABBCollision(Player, obstacle) && !
CheckTopCollisionOnly(Player, obstacle))
                    {
                        StopGame();
                        PlayerDied?.Invoke();
                        return;
                    }
                }
            }
        }

        private bool CheckAABBCollision(Player player, Obstacle obstacle)
        {
            return player.X < obstacle.X + obstacle.Width &&
                   player.X + PlayerSize > obstacle.X &&
                   player.Y < obstacle.Y + obstacle.Height &&
                   player.Y + PlayerSize > obstacle.Y;
        }

        private bool CheckTopCollisionOnly(Player player, Obstacle obstacle)
        {
            if (currentPositiveObstacle == obstacle) return true;

            if (player.VelocityY < 0) return false;

            if (player.Y + PlayerSize > obstacle.Y &&
                player.Y + PlayerSize < obstacle.Y + CollisionBuffer)
            {
```

```
                return true;
            }

            return false;
        }

        private Obstacle CheckTopLandingCollision()
        {
            foreach (var obstacle in Obstacles.Where(o => o.Type ==
ObstacleType.Positive))
            {
                if (Player.VelocityY >= 0)
                {
                    bool horizontalOverlap = Player.X + PlayerSize > obstacle.X &&
Player.X < obstacle.X + obstacle.Width;


                    if (horizontalOverlap &&
                        Player.Y + PlayerSize >= obstacle.Y &&
                        Player.Y + PlayerSize <= obstacle.Y + Player.VelocityY + 1)
                    {
                        return obstacle;
                    }
                }
            }
            return null;
        }


        public void StartGame()
        {
            gameTimer.Start();
            isRunning = true;
        }

        public void PauseGame()
        {
            gameTimer.Stop();
            isRunning = false;
        }

        public void ResumeGame()
        {
            gameTimer.Start();
            isRunning = true;
        }

        public void StopGame()
        {
            gameTimer.Stop();
            isRunning = false;
        }

        public void RestartGame()
        {
            StopGame();

            Player.X = 50;
            Player.Y = GroundY;
            Player.IsJumping = false;
            Player.VelocityY = 0;
            currentPositiveObstacle = null;
```

```
            LoadLevel();

            StartGame();
        }

        public void Jump(object parameter)
        {
            if (!Player.IsJumping)
            {
                Player.IsJumping = true;
                Player.VelocityY = -jumpPower;
            }
        }

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged([CallerMemberName] string propertyName =
null)
        {
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
        }
    }
}
```

Додаток В

Views

GameOptionsWindow.xaml:

```
<Window x:Class="GeometryDash.Views.GameOptionsWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Game Options" Height="450" Width="800">
    <Grid Margin="1,0,-1,0">
        <Image Source="pack://application:,,,/Resources/logo1.jpg"
Stretch="UniformToFill"/>
        <StackPanel VerticalAlignment="Center" HorizontalAlignment="Center">
            <Border CornerRadius="5" BorderThickness="2" BorderBrush="Black"
Margin="0,0,0,5">
                <Button Content="Return to Menu" Margin="2" Width="150" Height="50"
Background="Aqua" FontSize="16" FontWeight="Bold" Click="ReturnToMenu_Click"/>
            </Border>
            <Border CornerRadius="5" BorderThickness="2" BorderBrush="Black"
Margin="0,0,0,5">
                <Button Content="Continue" Margin="2" Width="150" Height="50"
Background="LightYellow" FontWeight="Bold" FontSize="16" Click="ContinueGame_Click"/>
            </Border>
            <Border CornerRadius="5" BorderThickness="2" BorderBrush="Black"
Margin="0,0,0,5">
                <Button Content="Exit Game" Margin="2" Width="150" Height="50"
Background="Red" FontSize="16" FontWeight="Bold" Click="ExitGame_Click"/>
            </Border>
        </StackPanel>
    </Grid>
</Window>
```

GameOptionsWindow.xaml.cs:

```
using System.Windows;

namespace GeometryDash.Views
{
    public partial class GameOptionsWindow : Window
    {
        public GameOptionsWindow()
        {
            InitializeComponent();
        }

        private void ReturnToMenu_Click(object sender, RoutedEventArgs e)
        {
            MainMenu mainMenu = new MainMenu();
            mainMenu.Show();
            Application.Current.MainWindow.Close();
            this.Close();
        }

        private void ContinueGame_Click(object sender, RoutedEventArgs e)
        {
            this.Close();
        }

        private void ExitGame_Click(object sender, RoutedEventArgs e)
        {
            Application.Current.Shutdown();
        }
    }
}
```

Level1Window.xaml:

```
<Window x:Class="GeometryDash.Views.Level1Window"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:GeometryDash.Views"
        mc:Ignorable="d"
        Title="Level1Window" Height="450" Width="1500">
    <Grid>
        <MediaElement Name="BackgroundMusic" Source="/Resources/Звук Dry Out.mp3"
LoadedBehavior="Manual" UnloadedBehavior="Stop" />

        <Image Source="/Resources/logo13.jpg" Stretch="UniformToFill"/>
        <Canvas Name="GameCanvas" Margin="0,0,0,10"
RenderTransformOrigin="0.823,0.561">
            <Image Name="PlayerSprite" Height="30" Canvas.Left="44" Canvas.Top="200"
Width="30" Source="/Views/logo4.png" HorizontalAlignment="Left"
VerticalAlignment="Top"/>
            <Rectangle Name="Obstacle1" Fill="Aqua" Width="30" Height="30"
Canvas.Left="400" Canvas.Top="200"/>
            <Polygon Name="Obstacle2" Fill="DeepPink" Points="0,30 15,0 30,30"
Canvas.Left="600" Canvas.Top="200" />
            <Button Name="Menu" Width="60" Height="27" Background="Aquamarine"
Canvas.Left="10" Click="Menu_Click" HorizontalAlignment="Left" VerticalAlignment="Top"
Content="Menu" />
            <Image Name="Logo" Source="/Views/logo7-Photoroom.png" Width="185"
Height="41" HorizontalAlignment="Left" VerticalAlignment="Top" Canvas.Left="10"
Canvas.Top="388"/>
```

```
            <Rectangle Name="PositiveObstacle1" Fill="DarkViolet" Width="57"
Height="58" Canvas.Left="821" Canvas.Top="168" RenderTransformOrigin="1.483,0.768"
HorizontalAlignment="Center" VerticalAlignment="Top"/>

            <Polygon Name="Obstacle4" Fill="Red" Points="0,30 15,0 30,30"
Canvas.Left="1110" Canvas.Top="200" HorizontalAlignment="Left"
VerticalAlignment="Center"/>

            <Rectangle Name="PositiveObstacle2" Fill="LightCoral" Width="95"
Height="50" Canvas.Left="1301" Canvas.Top="180" HorizontalAlignment="Center"
VerticalAlignment="Top"/>
            <Rectangle Name="PositiveObstacle3" Fill="LightCoral" Width="94"
Height="102" Canvas.Left="1453" Canvas.Top="128" HorizontalAlignment="Center"
VerticalAlignment="Top"/>
            <Rectangle Name="PositiveObstacle4" Fill="LightCoral" Width="95"
Height="159" Canvas.Left="1591" Canvas.Top="71" HorizontalAlignment="Center"
VerticalAlignment="Top"/>
            <Rectangle Name="PositiveObstacle5" Fill="Violet" Width="88" Height="159"
Canvas.Left="1775" Canvas.Top="71" HorizontalAlignment="Center"
VerticalAlignment="Top"/>

        </Canvas>
    </Grid>
</Window>
```

Level1Window.xaml.cs:

```
using GeometryDash.Models;
using System;
using System.Windows;
using System.Windows.Media;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Shapes;
using System.Windows.Threading;

namespace GeometryDash.Views
{
    public partial class Level1Window : Window
    {
        private DispatcherTimer gameTimer;
        private bool isJumping = false;
        private double gravity = 1;
        private double jumpStrength = -13.5;
        private double verticalVelocity = 0;
        private double obstacleSpeed = 5;
        private bool gameOver = false;
        private DateTime startTime;
        private bool isFalling = false;
        private bool isOnPositiveObstacle = false;
        private double positiveObstacleSpeedMultiplier = 0.3;
```

```
        private Rectangle currentPositiveObstacle = null;
        private MediaPlayer gameMusic;

        public Level1Window()
        {
            InitializeComponent();

            if (BackgroundMusic.Source != null)
            {
                BackgroundMusic.Play();
            }
            else
            {
                MessageBox.Show("Файл музики не знайдено. Перевірте шлях до файлу.");
            }

            gameTimer = new DispatcherTimer();
            gameTimer.Interval = TimeSpan.FromMilliseconds(15);
            gameTimer.Tick += GameLoop;
            gameTimer.Start();

            this.KeyDown += Level1Window_KeyDown;

            startTime = DateTime.Now;
            gameMusic = new MediaPlayer();
            gameMusic.Open(new Uri("D:\\Course work\\GeometryDash\\GeometryDash\\
Resources\\Звук Dry Out.mp3"));
            gameMusic.Play();
        }

        private void Level1Window_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.Key == Key.Space && !isJumping)
            {
                isJumping = true;
                verticalVelocity = jumpStrength;
            }
            else if (e.Key == Key.LeftShift || e.Key == Key.RightShift)
            {
                ShowGameOptions();
            }
        }
```

```
        private void GameLoop(object sender, EventArgs e)
        {
            if (gameOver)
            {
                RestartGame();
                return;
            }

            if (isJumping)
            {
                Canvas.SetTop(PlayerSprite, Canvas.GetTop(PlayerSprite) +
verticalVelocity);
                verticalVelocity += gravity;

                if (verticalVelocity > 0)
                {
                    isFalling = true;
                }

                if (CheckLandingOnPositiveObstacle())
                {
                    isJumping = false;
                    isFalling = false;
                    verticalVelocity = 0;
                    isOnPositiveObstacle = true;
                }
                else if (Canvas.GetTop(PlayerSprite) >= 200)
                {
                    Canvas.SetTop(PlayerSprite, 200);
                    isJumping = false;
                    isFalling = false;
                    verticalVelocity = 0;
                }
            }

            if (isOnPositiveObstacle)
            {
                double playerLeft = Canvas.GetLeft(PlayerSprite);
                double obstacleLeft = Canvas.GetLeft(currentPositiveObstacle);
                double obstacleRight = obstacleLeft +
currentPositiveObstacle.ActualWidth;
```

```
                if (playerLeft + PlayerSprite.ActualWidth / 2 > obstacleRight)
                {
                    isOnPositiveObstacle = false;
                    isFalling = true;
                    isJumping = true;
                    currentPositiveObstacle = null;
                }
                else
                {
                    Canvas.SetLeft(PlayerSprite, playerLeft + obstacleSpeed *
positiveObstacleSpeedMultiplier);
                }
            }

            MoveObstacle(Obstacle1);
            MoveObstacle(Obstacle2);
            MoveObstacle(Obstacle4);
            MoveObstacle(PositiveObstacle1);
            MoveObstacle(PositiveObstacle2);
            MoveObstacle(PositiveObstacle3);
            MoveObstacle(PositiveObstacle4);
            MoveObstacle(PositiveObstacle5);

            if (CheckCollision(PlayerSprite, Obstacle1) ||
                CheckCollision(PlayerSprite, Obstacle2) ||
                CheckCollision(PlayerSprite, Obstacle4))
            {
                gameOver = true;
            }

            if (CheckDestructivePositiveCollision(PositiveObstacle1) ||
                CheckDestructivePositiveCollision(PositiveObstacle2) ||
                CheckDestructivePositiveCollision(PositiveObstacle3) ||
                CheckDestructivePositiveCollision(PositiveObstacle4) ||
                CheckDestructivePositiveCollision(PositiveObstacle5))
            {
                gameOver = true;
            }

            if (Canvas.GetLeft(PlayerSprite) > Canvas.GetLeft(PositiveObstacle5) +
PositiveObstacle5.Width)
```

```
        {
            CompleteLevel();
        }
    }

    private bool CheckDestructivePositiveCollision(Rectangle obstacle)
    {
        const double PlayerHeight = 30;
        const double SideCollisionTolerance = 5;

        if (CheckCollision(PlayerSprite, obstacle))
        {
            double playerTop = Canvas.GetTop(PlayerSprite);
            double playerBottom = playerTop + PlayerHeight;
            double obstacleTop = Canvas.GetTop(obstacle);

            if (verticalVelocity < 0)
            {
                return true;
            }

            if (playerBottom > obstacleTop + SideCollisionTolerance)
            {
                return true;
            }

            if (isOnPositiveObstacle && currentPositiveObstacle == obstacle)
            {
                return false;
            }
        }
        return false;
    }

    private void MoveObstacle(FrameworkElement obstacle)
    {
        double left = Canvas.GetLeft(obstacle);
        double width = obstacle.Width;

        if (width <= 0)
        {
            if (obstacle.ActualWidth > 0)
```

```
                width = obstacle.ActualWidth;
            else if (obstacle is Polygon poly)
            {
                double minX = double.MaxValue, maxX = double.MinValue;
                foreach (var p in poly.Points)
                {
                    if (p.X < minX) minX = p.X;
                    if (p.X > maxX) maxX = p.X;
                }
                width = (maxX - minX);
            }

            if (width <= 0) width = 30;
        }

        if (left < -width)
            left = GameCanvas.ActualWidth;
        else
            left -= obstacleSpeed;

        Canvas.SetLeft(obstacle, left);
    }

    private bool CheckLandingOnPositiveObstacle()
    {
        return CheckLanding(PlayerSprite, PositiveObstacle1) ||
                CheckLanding(PlayerSprite, PositiveObstacle2) ||
                CheckLanding(PlayerSprite, PositiveObstacle3) ||
                CheckLanding(PlayerSprite, PositiveObstacle4) ||
                CheckLanding(PlayerSprite, PositiveObstacle5);
    }

    private bool CheckLanding(Image player, Rectangle obstacle)
    {
        const double LandingTolerance = 15;

        if (CheckCollision(player, obstacle))
        {
            double playerBottom = Canvas.GetTop(player) + player.ActualHeight;
            double obstacleTop = Canvas.GetTop(obstacle);

            if (verticalVelocity >= 0 &&
```

```
                    playerBottom >= obstacleTop &&
                    playerBottom <= obstacleTop + LandingTolerance)
                {
                    Canvas.SetTop(player, obstacleTop - player.ActualHeight);

                    currentPositiveObstacle = obstacle;
                    isOnPositiveObstacle = true;
                    return true;
                }
            }

            return false;
        }


        private bool CheckCollision(Image player, Shape obstacle)
        {
            Rect playerRect = new Rect(Canvas.GetLeft(player), Canvas.GetTop(player),
player.Width, player.Height);

            Rect obstacleRect;

            if (obstacle is Polygon poly)
            {
                if (poly.Points == null || poly.Points.Count == 0)
                {
                    obstacleRect = new Rect(Canvas.GetLeft(poly), Canvas.GetTop(poly),
30, 30);
                }
                else
                {
                    double minX = double.MaxValue, minY = double.MaxValue;
                    double maxX = double.MinValue, maxY = double.MinValue;
                    foreach (var p in poly.Points)
                    {
                        if (p.X < minX) minX = p.X;
                        if (p.Y < minY) minY = p.Y;
                        if (p.X > maxX) maxX = p.X;
                        if (p.Y > maxY) maxY = p.Y;
                    }

                    double left = Canvas.GetLeft(poly) + minX;
```

```
                    double top = Canvas.GetTop(poly) + minY;
                    double width = Math.Max(1, maxX - minX);
                    double height = Math.Max(1, maxY - minY);

                    obstacleRect = new Rect(left, top, width, height);
                }
            }
            else
            {
                double width = obstacle.Width > 0 ? obstacle.Width :
obstacle.ActualWidth;
                double height = obstacle.Height > 0 ? obstacle.Height :
obstacle.ActualHeight;
                if (width <= 0) width = 30;
                if (height <= 0) height = 30;

                obstacleRect = new Rect(Canvas.GetLeft(obstacle),
Canvas.GetTop(obstacle), width, height);
            }

            return playerRect.IntersectsWith(obstacleRect);
        }

        private bool CheckCollision(Image player, Rectangle obstacle)
        {
            Rect playerRect = new Rect(Canvas.GetLeft(player), Canvas.GetTop(player),
player.Width, player.Height);
            Rect obstacleRect = new Rect(Canvas.GetLeft(obstacle),
Canvas.GetTop(obstacle), obstacle.Width, obstacle.Height);
            return playerRect.IntersectsWith(obstacleRect);
        }



        public void RestartGame()
        {
            Canvas.SetLeft(PlayerSprite, 44);
            Canvas.SetTop(PlayerSprite, 200);

            Canvas.SetLeft(Obstacle1, 400);
            Canvas.SetTop(Obstacle1, 200);
```

```
        Canvas.SetLeft(Obstacle2, 600);
        Canvas.SetTop(Obstacle2, 200);

        Canvas.SetLeft(Obstacle4, 1065);
        Canvas.SetTop(Obstacle4, 202);

        Canvas.SetLeft(PositiveObstacle1, 821);
        Canvas.SetTop(PositiveObstacle1, 168);

        Canvas.SetLeft(PositiveObstacle2, 1301);
        Canvas.SetTop(PositiveObstacle2, 180);

        Canvas.SetLeft(PositiveObstacle3, 1453);
        Canvas.SetTop(PositiveObstacle3, 128);

        Canvas.SetLeft(PositiveObstacle4, 1591);
        Canvas.SetTop(PositiveObstacle4, 71);

        Canvas.SetLeft(PositiveObstacle5, 1775);
        Canvas.SetTop(PositiveObstacle5, 71);

        isJumping = false;
        verticalVelocity = 0;
        gameOver = false;
        startTime = DateTime.Now;
        isOnPositiveObstacle = false;
        currentPositiveObstacle = null;

        gameMusic.Stop();
        gameMusic.Position = TimeSpan.Zero;
        gameMusic.Play();

        gameTimer.Start();
    }

    private void Menu_Click(object sender, RoutedEventArgs e)
    {
        ShowGameOptions();
    }

    private void ShowGameOptions()
    {
```

```
            GameOptionsWindow optionsWindow = new GameOptionsWindow();
            optionsWindow.Owner = this;
            optionsWindow.ShowDialog();
        }

        private void CompleteLevel()
        {
            gameTimer.Stop();
            gameMusic.Stop();

            LevelCompleteWindow levelCompleteWindow = new LevelCompleteWindow
            {
                CompletedLevel = "Level1"
            };

            levelCompleteWindow.Show();
            this.Close();
        }



    }
}
```

Level2Window.xaml:

```
<Window x:Class="GeometryDash.Views.Level2Window"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:GeometryDash.Views"
        mc:Ignorable="d"
        Title="Level2Window" Height="450" Width="1500">
    <Grid>
        <MediaElement Name="BackgroundMusic" Source="/Resources/Звук Dry Out.mp3"
LoadedBehavior="Manual" UnloadedBehavior="Stop" />

        <Image Source="/Resources/logo12.jpg" Stretch="UniformToFill"/>
        <Canvas Name="GameCanvas" Margin="0,0,0,10"
RenderTransformOrigin="0.823,0.561">
            <Image Name="PlayerSprite" Height="30" Canvas.Left="44" Canvas.Top="200"
Width="30" Source="/Views/logo4.png" HorizontalAlignment="Left"
VerticalAlignment="Top"/>
            <Rectangle Name="Obstacle1" Fill="Aqua" Width="30" Height="30"
Canvas.Left="331" Canvas.Top="200" HorizontalAlignment="Left"
VerticalAlignment="Center"/>
            <Polygon Name="Obstacle2" Fill="DeepPink" Points="0,30 15,0 30,30"
Canvas.Left="789" Canvas.Top="200" />
```

```
            <Button Name="Menu" Width="60" Height="27" Background="Aquamarine"
Canvas.Left="10" Click="Menu_Click" HorizontalAlignment="Left" VerticalAlignment="Top"
Content="Menu" />
            <Image Name="Logo" Source="/Views/logo7-Photoroom.png" Width="185"
Height="41" HorizontalAlignment="Left" VerticalAlignment="Top" Canvas.Left="10"
Canvas.Top="388"/>
            <Rectangle Name="PositiveObstacle1" Fill="DarkViolet" Width="57"
Height="64" Canvas.Left="550" Canvas.Top="166" RenderTransformOrigin="0.523,0.538"
HorizontalAlignment="Center" VerticalAlignment="Top"/>

            <Polygon Name="Obstacle4" Fill="Red" Points="0,30 15,0 30,30"
Canvas.Left="1338" Canvas.Top="202" />

            <Rectangle Name="PositiveObstacle2" Fill="LightCoral" Width="95"
Height="68" Canvas.Left="1019" Canvas.Top="162" HorizontalAlignment="Center"
VerticalAlignment="Top"/>
            <Rectangle Name="PositiveObstacle3" Fill="LightCoral" Width="94"
Height="71" Canvas.Left="1604" Canvas.Top="162" HorizontalAlignment="Center"
VerticalAlignment="Top"/>
            <Rectangle Name="PositiveObstacle4" Fill="LightCoral" Width="95"
Height="38" Canvas.Left="1722" Canvas.Top="125" HorizontalAlignment="Center"
VerticalAlignment="Top"/>
            <Rectangle Name="PositiveObstacle5" Fill="Violet" Width="88" Height="30"
Canvas.Left="1870" Canvas.Top="200" HorizontalAlignment="Left"
VerticalAlignment="Top"/>


        </Canvas>
    </Grid>
</Window>
```

Level2Window.xaml.cs:

```
using System;
using System.Windows;
using System.Windows.Media;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Shapes;
using System.Windows.Threading;

namespace GeometryDash.Views
{
    public partial class GameOptionsWindow : Window
    {
        public bool ShouldNavigateToMainMenu { get; set; } = false;
    }

    public partial class Level2Window : Window
    {
        private DispatcherTimer gameTimer;
        private bool isJumping = false;
        private double gravity = 1;
```

```
        private double jumpStrength = -12;
        private double verticalVelocity = 0;
        private double obstacleSpeed = 7;
        private bool gameOver = false;
        private DateTime startTime;
        private bool isFalling = false;
        private bool isOnPositiveObstacle = false;
        private double positiveObstacleSpeedMultiplier = 0.3;
        private Rectangle currentPositiveObstacle = null;
        private MediaPlayer gameMusic;


        public Level2Window()
        {
            InitializeComponent();

            if (BackgroundMusic.Source != null)
            {
                BackgroundMusic.Play();
            }
            else
            {
                MessageBox.Show("Файл музики не знайдено. Перевірте шлях до файлу.");
            }

            gameTimer = new DispatcherTimer();
            gameTimer.Interval = TimeSpan.FromMilliseconds(15);
            gameTimer.Tick += GameLoop;
            gameTimer.Start();

            this.KeyDown += Level2Window_KeyDown;
            this.Closed += Level2Window_Closed;

            startTime = DateTime.Now;
            gameMusic = new MediaPlayer();
            gameMusic.Open(new Uri("D:\\Course work\\GeometryDash\\GeometryDash\\
Resources\\Forever Bound Stereo Madness (Geometry Dash).mp3"));
            gameMusic.Play();

        }

        private void Level2Window_Closed(object sender, EventArgs e)
```

```
        {
            if (gameMusic != null)
            {
                gameMusic.Stop();
            }
            if (gameTimer != null)
            {
                gameTimer.Stop();
            }
        }

        private void Level2Window_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.Key == Key.Space && !isJumping)
            {
                isJumping = true;
                verticalVelocity = jumpStrength;
            }
            else if (e.Key == Key.LeftShift || e.Key == Key.RightShift)
            {
                ShowGameOptions();
            }
        }

        private void GameLoop(object sender, EventArgs e)
        {
            if (gameOver)
            {
                RestartGame();
                return;
            }

            if (isJumping)
            {
                Canvas.SetTop(PlayerSprite, Canvas.GetTop(PlayerSprite) +
verticalVelocity);
                verticalVelocity += gravity;

                if (verticalVelocity > 0)
                {
                    isFalling = true;
                }
```

```
                if (CheckLandingOnPositiveObstacle())
                {
                    isJumping = false;
                    isFalling = false;
                    verticalVelocity = 0;
                    isOnPositiveObstacle = true;
                }
                else if (Canvas.GetTop(PlayerSprite) >= 200)
                {
                    Canvas.SetTop(PlayerSprite, 200);
                    isJumping = false;
                    isFalling = false;
                    verticalVelocity = 0;
                }
            }

            if (isOnPositiveObstacle)
            {
                double playerLeft = Canvas.GetLeft(PlayerSprite);
                double obstacleLeft = Canvas.GetLeft(currentPositiveObstacle);
                double obstacleRight = obstacleLeft +
currentPositiveObstacle.ActualWidth;

                if (playerLeft + PlayerSprite.ActualWidth / 2 > obstacleRight)
                {
                    isOnPositiveObstacle = false;
                    isFalling = true;
                    isJumping = true;
                    currentPositiveObstacle = null;
                }
                else
                {
                    Canvas.SetLeft(PlayerSprite, playerLeft + obstacleSpeed *
positiveObstacleSpeedMultiplier);
                }
            }

            MoveObstacle(Obstacle1);
            MoveObstacle(Obstacle2);
            MoveObstacle(Obstacle4);
            MoveObstacle(PositiveObstacle1);
```

```
        MoveObstacle(PositiveObstacle2);
        MoveObstacle(PositiveObstacle3);
        MoveObstacle(PositiveObstacle4);
        MoveObstacle(PositiveObstacle5);

        if (CheckCollision(PlayerSprite, Obstacle1) ||
            CheckCollision(PlayerSprite, Obstacle2) ||
            CheckCollision(PlayerSprite, Obstacle4) ||
            CheckCollision(PlayerSprite, PositiveObstacle1) ||
            CheckCollision(PlayerSprite, PositiveObstacle2) ||
            CheckCollision(PlayerSprite, PositiveObstacle3) ||
            CheckCollision(PlayerSprite, PositiveObstacle4) ||
            CheckCollision(PlayerSprite, PositiveObstacle5))
        {
            gameOver = true;
        }


        if (Canvas.GetLeft(PlayerSprite) > Canvas.GetLeft(PositiveObstacle5))
        {
            CompleteLevel();
        }
    }

    private void MoveObstacle(Shape obstacle)
    {
        double left = Canvas.GetLeft(obstacle);
        if (left < -obstacle.ActualWidth)
        {
            left = GameCanvas.ActualWidth;
        }
        else
        {
            left -= obstacleSpeed;
        }
        Canvas.SetLeft(obstacle, left);
    }

    private bool CheckLandingOnPositiveObstacle()
    {
        return CheckLanding(PlayerSprite, PositiveObstacle1) ||
                   CheckLanding(PlayerSprite, PositiveObstacle2) ||
```

```
                        CheckLanding(PlayerSprite, PositiveObstacle3) ||
                        CheckLanding(PlayerSprite, PositiveObstacle4) ||
                        CheckLanding(PlayerSprite, PositiveObstacle5);
        }

        private bool CheckLanding(Image player, Rectangle obstacle)
        {
            Rect playerRect = new Rect(Canvas.GetLeft(player), Canvas.GetTop(player),
player.Width, player.Height);
            Rect obstacleRect = new Rect(Canvas.GetLeft(obstacle),
Canvas.GetTop(obstacle), obstacle.ActualWidth, obstacle.ActualHeight);

            bool isAbovePlatform = playerRect.Bottom <= obstacleRect.Top + 5 &&
                                   playerRect.Right > obstacleRect.Left + 5 &&
                                   playerRect.Left < obstacleRect.Right - 5 &&
                                   verticalVelocity >= 0;

            if (isAbovePlatform && playerRect.Bottom + verticalVelocity >=
obstacleRect.Top)
            {
                Canvas.SetTop(player, obstacleRect.Top - player.Height);
                currentPositiveObstacle = obstacle;
                isOnPositiveObstacle = true;
                isJumping = false;
                isFalling = false;
                verticalVelocity = 0;
                return true;
            }

            return false;
        }


        private bool CheckCollision(Image player, Shape obstacle)
        {
            Rect playerRect = new Rect(Canvas.GetLeft(player), Canvas.GetTop(player),
player.Width, player.Height);
            Rect obstacleRect = new Rect(Canvas.GetLeft(obstacle),
Canvas.GetTop(obstacle), obstacle.ActualWidth, obstacle.ActualHeight);

            if (obstacle.Name != null && obstacle.Name.StartsWith("PositiveObstacle"))
            {
```

```
            Rect dangerousObstacleRect = new Rect(
                obstacleRect.Left,
                obstacleRect.Top + 1,
                obstacleRect.Width,
                obstacleRect.Height - 1
            );

            if (playerRect.IntersectsWith(dangerousObstacleRect))
            {
                return true;
            }
            return false;
        }

        return playerRect.IntersectsWith(obstacleRect);
    }

    public void RestartGame()
    {
        Canvas.SetLeft(PlayerSprite, 44);
        Canvas.SetTop(PlayerSprite, 200);

        Canvas.SetLeft(Obstacle1, 331);
        Canvas.SetTop(Obstacle1, 200);

        Canvas.SetLeft(Obstacle2, 789);
        Canvas.SetTop(Obstacle2, 200);

        Canvas.SetLeft(Obstacle4, 1338);
        Canvas.SetTop(Obstacle4, 202);

        Canvas.SetLeft(PositiveObstacle1, 550);
        Canvas.SetTop(PositiveObstacle1, 166);

        Canvas.SetLeft(PositiveObstacle2, 1019);
        Canvas.SetTop(PositiveObstacle2, 162);

        Canvas.SetLeft(PositiveObstacle3, 1604);
        Canvas.SetTop(PositiveObstacle3, 162);

        Canvas.SetLeft(PositiveObstacle4, 1722);
        Canvas.SetTop(PositiveObstacle4, 125);
```

```
        Canvas.SetLeft(PositiveObstacle5, 1870);
        Canvas.SetTop(PositiveObstacle5, 200);
        isJumping = false;
        verticalVelocity = 0;
        gameOver = false;
        startTime = DateTime.Now;
        isOnPositiveObstacle = false;
        currentPositiveObstacle = null;

        gameMusic.Stop();
        gameMusic.Position = TimeSpan.Zero;
        gameMusic.Play();
        gameTimer.Start();
    }

    private void Menu_Click(object sender, RoutedEventArgs e)
    {
        ShowGameOptions();
    }

    private void Continue_Click(object sender, RoutedEventArgs e)
    {
        this.Close();
    }

    private void ShowGameOptions()
    {
        gameTimer.Stop();
        gameMusic.Pause();

        GameOptionsWindow optionsWindow = new GameOptionsWindow();
        optionsWindow.Owner = this;

        optionsWindow.ShowDialog();

        if (optionsWindow.ShouldNavigateToMainMenu)
        {
            NavigateToMainMenu();
        }
        else if (!gameOver)
        {
```

```
                gameTimer.Start();
                gameMusic.Play();
            }
        }

        public void NavigateToMainMenu()
        {
            gameTimer.Stop();
            gameMusic.Stop();

            try
            {
                MainMenu mainMenu = new MainMenu();
                mainMenu.Show();
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Помилка відкриття головного меню: {ex.Message}");
            }

            this.Close();
        }


        private void CompleteLevel()
        {
            gameTimer.Stop();

            LevelCompleteWindow levelCompleteWindow = new LevelCompleteWindow
            {
                CompletedLevel = "Level2"
            };

            levelCompleteWindow.Show();
            this.Close();
        }

    }
}
```

Level3Window.xaml:

```
<Window x:Class="GeometryDash.Views.Level3Window"
```

```
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:GeometryDash.Views"
        mc:Ignorable="d"
        Title="Level3Window" Height="450" Width="2000">
    <Grid>
        <MediaElement Name="BackgroundMusic" Source="/Resources/Звук Dry Out.mp3"
LoadedBehavior="Manual" UnloadedBehavior="Stop" />

        <Image Source="/Resources/logo17.jpg" Stretch="UniformToFill" Margin="0,0,-
2109,0"/>
        <Canvas Name="GameCanvas" Margin="0,0,0,10"
RenderTransformOrigin="0.823,0.561">
            <Image Name="PlayerSprite" Height="30" Canvas.Left="44" Canvas.Top="200"
Width="30" Source="/Views/logo4.png" HorizontalAlignment="Left"
VerticalAlignment="Top"/>

            <Polygon Name="Obstacle1" Fill="Aqua" Points="0,30 15,0 30,30"
Canvas.Left="683" Canvas.Top="200" />
            <Polygon Name="Obstacle2" Fill="DeepPink" Points="0,30 15,0 30,30"
Canvas.Left="350" Canvas.Top="200" />

            <Button Name="Menu" Width="60" Height="27" Background="Aquamarine"
Canvas.Left="10" Click="Menu_Click" HorizontalAlignment="Left" VerticalAlignment="Top"
Content="Menu" />
            <Image Name="Logo" Source="/Views/logo7-Photoroom.png" Width="185"
Height="41" HorizontalAlignment="Center" VerticalAlignment="Top" Canvas.Left="10"
Canvas.Top="378"/>
            <Rectangle Name="PositiveObstacle1" Fill="DarkViolet" Width="57"
Height="132" Canvas.Left="1579" Canvas.Top="95" RenderTransformOrigin="1.483,0.768"
HorizontalAlignment="Center" VerticalAlignment="Center"/>


            <Rectangle Name="PositiveObstacle2" Fill="LightCoral" Width="96"
Height="80" Canvas.Left="1042" Canvas.Top="150" HorizontalAlignment="Center"
VerticalAlignment="Top"/>
            <Rectangle Name="PositiveObstacle3" Fill="LightCoral" Width="94"
Height="132" Canvas.Left="1281" Canvas.Top="95" HorizontalAlignment="Center"
VerticalAlignment="Top"/>
            <Rectangle Name="PositiveObstacle5" Fill="Violet" Width="88" Height="75"
Canvas.Left="1882" Canvas.Top="152" HorizontalAlignment="Center"
VerticalAlignment="Top"/>


        </Canvas>
    </Grid>
</Window>
```

Level3Window.xaml.cs:

```
using System;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Shapes;
using System.Windows.Threading;

namespace GeometryDash.Views
{
```

```
    public partial class Level3Window : Window
    {
        private DispatcherTimer gameTimer;
        private bool isJumping = false;
        private double gravity = 1;
        private double jumpStrength = -13;
        private double verticalVelocity = 0;
        private double obstacleSpeed = 10;
        private bool gameOver = false;
        private bool isFalling = false;
        private bool isOnPositiveObstacle = false;
        private double positiveObstacleSpeedMultiplier = 0.3;
        private Rectangle currentPositiveObstacle = null;
        private MediaPlayer gameMusic;
        private DateTime startTime;

        public Level3Window()
        {
            InitializeComponent();

            if (BackgroundMusic.Source != null)
                BackgroundMusic.Play();
            else
                MessageBox.Show("Файл музики не знайдено. Перевірте шлях до файлу.");

            gameTimer = new DispatcherTimer();
            gameTimer.Interval = TimeSpan.FromMilliseconds(15);
            gameTimer.Tick += GameLoop;
            gameTimer.Start();

            this.KeyDown += Level3Window_KeyDown;

            startTime = DateTime.Now;
            gameMusic = new MediaPlayer();
            try
            {
                gameMusic.Open(new Uri("D:\\Course work\\GeometryDash\\GeometryDash\\
Resources\\Geometry_Dash-Back_On_Track-world76.spcs.bio.mp3"));
                gameMusic.Play();
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Помилка завантаження музики: {ex.Message}");
            }
        }

        private void Level3Window_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.Key == Key.Space && !isJumping)
            {
                isJumping = true;
                verticalVelocity = jumpStrength;
            }
            else if (e.Key == Key.LeftShift || e.Key == Key.RightShift)
            {
                ShowGameOptions();
            }
        }

        private void GameLoop(object sender, EventArgs e)
        {
            if (gameOver)
            {
```

```
                RestartGame();
                return;
            }

            if (isJumping)
            {
                Canvas.SetTop(PlayerSprite, Canvas.GetTop(PlayerSprite) +
verticalVelocity);
                verticalVelocity += gravity;

                if (verticalVelocity > 0)
                    isFalling = true;

                if (CheckLandingOnPositiveObstacle())
                {
                    isJumping = false;
                    isFalling = false;
                    verticalVelocity = 0;
                    isOnPositiveObstacle = true;
                }
                else if (Canvas.GetTop(PlayerSprite) >= 200)
                {
                    Canvas.SetTop(PlayerSprite, 200);
                    isJumping = false;
                    isFalling = false;
                    verticalVelocity = 0;
                }
            }

            if (isOnPositiveObstacle)
            {
                double playerLeft = Canvas.GetLeft(PlayerSprite);
                double obstacleLeft = Canvas.GetLeft(currentPositiveObstacle);
                double obstacleRight = obstacleLeft +
currentPositiveObstacle.ActualWidth;

                if (playerLeft + PlayerSprite.ActualWidth / 2 > obstacleRight)
                {
                    isOnPositiveObstacle = false;
                    isFalling = true;
                    isJumping = true;
                    currentPositiveObstacle = null;
                }
                else
                {
                    Canvas.SetLeft(PlayerSprite, playerLeft + obstacleSpeed *
positiveObstacleSpeedMultiplier);
                }
            }

            MoveObstacle(Obstacle1);
            MoveObstacle(Obstacle2);
            MoveObstacle(PositiveObstacle1);
            MoveObstacle(PositiveObstacle2);
            MoveObstacle(PositiveObstacle3);
            MoveObstacle(PositiveObstacle5);

            if (CheckCollisionWithDanger(PlayerSprite, Obstacle1) ||
                CheckCollisionWithDanger(PlayerSprite, Obstacle2) ||
                CheckCollisionWithDanger(PlayerSprite, PositiveObstacle1) ||
                CheckCollisionWithDanger(PlayerSprite, PositiveObstacle2) ||
                CheckCollisionWithDanger(PlayerSprite, PositiveObstacle3) ||
                CheckCollisionWithDanger(PlayerSprite, PositiveObstacle5))
```

```
            {
                gameOver = true;
            }

            if (Canvas.GetLeft(PlayerSprite) > Canvas.GetLeft(PositiveObstacle5) +
PositiveObstacle5.Width)
            {
                CompleteLevel();
            }
        }

        private void MoveObstacle(Shape obstacle)
        {
            double left = Canvas.GetLeft(obstacle);
            if (left < -obstacle.Width)
                left = GameCanvas.ActualWidth;
            else
                left -= obstacleSpeed;
            Canvas.SetLeft(obstacle, left);
        }

        private bool CheckLandingOnPositiveObstacle()
        {
            return CheckLanding(PlayerSprite, PositiveObstacle1) ||
                       CheckLanding(PlayerSprite, PositiveObstacle2) ||
                       CheckLanding(PlayerSprite, PositiveObstacle3) ||
                       CheckLanding(PlayerSprite, PositiveObstacle5);
        }

        private bool CheckLanding(Image player, Rectangle obstacle)
        {
            Rect playerRect = new Rect(Canvas.GetLeft(player), Canvas.GetTop(player),
player.Width, player.Height);
            Rect obstacleRect = new Rect(Canvas.GetLeft(obstacle),
Canvas.GetTop(obstacle), obstacle.ActualWidth, obstacle.ActualHeight);

            bool isAbove = playerRect.Bottom <= obstacleRect.Top + 5 &&
                           playerRect.Right > obstacleRect.Left + 5 &&
                           playerRect.Left < obstacleRect.Right - 5 &&
                           verticalVelocity >= 0;

            if (isAbove && playerRect.Bottom + verticalVelocity >= obstacleRect.Top)
            {
                Canvas.SetTop(player, obstacleRect.Top - player.Height);
                currentPositiveObstacle = obstacle;
                isOnPositiveObstacle = true;
                isJumping = false;
                isFalling = false;
                verticalVelocity = 0;
                return true;
            }

            return false;
        }

        private bool CheckCollisionWithDanger(Image player, Shape obstacle)
        {
            Rect playerRect = new Rect(Canvas.GetLeft(player), Canvas.GetTop(player),
player.Width, player.Height);
            Rect obstacleRect = new Rect(Canvas.GetLeft(obstacle),
Canvas.GetTop(obstacle), obstacle.ActualWidth, obstacle.ActualHeight);

            if (!obstacle.Name.StartsWith("PositiveObstacle"))
```

```
            return playerRect.IntersectsWith(obstacleRect);

        Rect dangerousZone = new Rect(
            obstacleRect.Left,
            obstacleRect.Top + 5,
            obstacleRect.Width,
            obstacleRect.Height - 5
        );

        if (playerRect.IntersectsWith(dangerousZone))
            return true;

        return false;
    }

    public void RestartGame()
    {
        Canvas.SetLeft(PlayerSprite, 44);
        Canvas.SetTop(PlayerSprite, 200);

        Canvas.SetLeft(Obstacle1, 683);
        Canvas.SetTop(Obstacle1, 200);

        Canvas.SetLeft(Obstacle2, 350);
        Canvas.SetTop(Obstacle2, 200);

        Canvas.SetLeft(PositiveObstacle1, 1579);
        Canvas.SetTop(PositiveObstacle1, 95);

        Canvas.SetLeft(PositiveObstacle2, 1042);
        Canvas.SetTop(PositiveObstacle2, 150);

        Canvas.SetLeft(PositiveObstacle3, 1281);
        Canvas.SetTop(PositiveObstacle3, 95);

        Canvas.SetLeft(PositiveObstacle5, 1882);
        Canvas.SetTop(PositiveObstacle5, 152);

        isJumping = false;
        verticalVelocity = 0;
        gameOver = false;
        isOnPositiveObstacle = false;
        currentPositiveObstacle = null;
        startTime = DateTime.Now;

        try
        {
            gameMusic.Stop();
            gameMusic.Position = TimeSpan.Zero;
            gameMusic.Play();
        }
        catch { /* Ігнорувати помилки музики */ }

        gameTimer.Start();
    }

    private void Menu_Click(object sender, RoutedEventArgs e)
    {
        ShowGameOptions();
    }

    private void ShowGameOptions()
    {
```

```
            MessageBox.Show("Game options would be shown here.");
        }

        private void CompleteLevel()
        {
            gameTimer.Stop();
            try
            {
                if (gameMusic != null)
                    gameMusic.Stop();
            }
            catch { /* Ігнорувати помилки музики при зупинці */ }

            LevelCompleteWindow levelCompleteWindow = new LevelCompleteWindow
            {
                CompletedLevel = "Level3"
            };
            levelCompleteWindow.Show();

            this.Close();
        }
    }
}
```

LevelCompleteWindow.xaml:

```
<Window x:Class="GeometryDash.Views.LevelCompleteWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Level Complete" Height="618" Width="1229">
    <Grid>
        <TextBlock Text="Level Complete!" HorizontalAlignment="Center"
VerticalAlignment="Top" Margin="0,440,0,0" FontSize="24"/>

        <Image Source="/Resources/logo14.jpg" Stretch="UniformToFill"/>
        <TextBox TextWrapping="Wrap" Text=" Level Complete!" FontSize="24"
FontWeight="Bold" Height="40" Margin="514,147,515,415" Background="Ivory"/>
        <StackPanel VerticalAlignment="Center" HorizontalAlignment="Center"
Margin="0,100,0,0">
            <Button Content="Retry Level" Width="200" Height="40"
Background="LightGreen" FontSize="16" FontWeight="Bold" Click="RetryLevel_Click"
Cursor="SizeAll"/>
            <Button Content="Next Level" Width="200" Height="40" Background="HotPink"
FontSize="16" FontWeight="Bold" Click="NextLevel_Click" Cursor="SizeWE"/>
            <Button Content="Main Menu" Width="200" Height="40" Background="Purple"
FontSize="16" FontWeight="Bold" Click="MainMenu_Click"/>
            <Button Content="Exit Game" Width="200" Height="40" Background="Red"
FontSize="16" FontWeight="Bold" Click="ExitGame_Click" Cursor="ScrollNW"/>

        </StackPanel>
        <TextBox TextWrapping="Wrap" Text="      🌸🥳Level Complete!🥳🌸    "
FontSize="24" FontWeight="Bold" Margin="436,147,433,395" Background="LightCoral"
TextChanged="TextBox_TextChanged"/>
    </Grid>
</Window>
```

LevelCompleteWindow.xaml.cs:

```
    using System;
    using System.Windows;

namespace GeometryDash.Views
```

```
{
    public partial class LevelCompleteWindow : Window
    {
        public string CompletedLevel { get; set; }

        public LevelCompleteWindow()
        {
            InitializeComponent();
        }

        private void ExitGame_Click(object sender, RoutedEventArgs e)
        {
            this.Close();
            Application.Current.Shutdown();
        }

        private void MainMenu_Click(object sender, RoutedEventArgs e)
        {
            try
            {
                MainMenu mainMenu = new MainMenu();
                mainMenu.Show();

                this.Close();
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Помилка відкриття головного меню: {ex.Message}");
            }
        }

        private void RetryLevel_Click(object sender, RoutedEventArgs e)
        {
            try
            {
                if (CompletedLevel == "Level1")
                    new Level1Window().Show();
                else if (CompletedLevel == "Level2")
                    new Level2Window().Show();
                else if (CompletedLevel == "Level3")
                    new Level3Window().Show();

                this.Close();
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Помилка перезапуску рівня: {ex.Message}");
            }
        }

        private void NextLevel_Click(object sender, RoutedEventArgs e)
        {
            try
            {
                if (CompletedLevel == "Level1")
                    new Level2Window().Show();
                else if (CompletedLevel == "Level2")
                    new Level3Window().Show();
                else if (CompletedLevel == "Level3")
                    MessageBox.Show("Congratulations! You have passed all levels 🎉");

                this.Close();
            }
```

```
            catch (Exception ex)
            {
                MessageBox.Show($"Помилка переходу на наступний рівень:
{ex.Message}");
            }
        }

        private void TextBox_TextChanged(object sender,
System.Windows.Controls.TextChangedEventArgs e)
        {

        }
    }
}
```

MainMenu.xaml:

```
<Window x:Class="GeometryDash.Views.MainMenu"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Geometry Dash" Height="618" Width="1229">
    <Grid>
        <Image Source="pack://application:,,,/Resources/logo.jpg"
Stretch="UniformToFill"/>
        <StackPanel VerticalAlignment="Center" HorizontalAlignment="Center">
            <Border CornerRadius="5" BorderThickness="2" BorderBrush="Black" >
                <Button Content="Play Level 1" Margin="2" Width="150" Height="50"
Background="LimeGreen" FontSize="16" FontWeight="Bold"
Click="PlayLevel1Button_Click"/>
            </Border>
            <Border CornerRadius="5" BorderThickness="2" BorderBrush="Black">
                <Button Content="Play Level 2" Margin="2" Width="150" Height="50"
Background="LimeGreen" FontSize="16" FontWeight="Bold"
Click="PlayLevel2Button_Click"/>
            </Border>
            <Border CornerRadius="5" BorderThickness="2" BorderBrush="Black">
                <Button Content="Play Level 3" Margin="2" Width="150" Height="50"
Background="LimeGreen" FontSize="16" FontWeight="Bold"
Click="PlayLevel3Button_Click"/>
            </Border>

            <Border CornerRadius="5" BorderThickness="2" BorderBrush="Black">
                <Button Content="Exit" Margin="2" Width="150" Height="50"
Background="Red" FontSize="16" FontWeight="Bold" Click="ExitButton_Click"/>
            </Border>
        </StackPanel>
    </Grid>
</Window>
```

MainMenu.xaml.cs:

```
using System.Windows;

namespace GeometryDash.Views
{
    public partial class MainMenu : Window
    {
        public MainMenu()
        {
            InitializeComponent();
        }

        private void PlayLevel1Button_Click(object sender, RoutedEventArgs e)
```

```
        {
            Level1Window level1Window = new Level1Window();
            level1Window.Show();
            this.Close();
        }

        private void PlayLevel2Button_Click(object sender, RoutedEventArgs e)
        {
            Level2Window level2Window = new Level2Window();
            level2Window.Show();
            this.Close();
        }

        private void PlayLevel3Button_Click(object sender, RoutedEventArgs e)
        {
            Level3Window level3Window = new Level3Window();
            level3Window.Show();
            this.Close();
        }



        private void ExitButton_Click(object sender, RoutedEventArgs e)
        {
            Application.Current.Shutdown();
        }



    }
}
```

App.xaml:

```
<Application x:Class="GeometryDash.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             StartupUri="Views/MainMenu.xaml">
    <Application.Resources>

    </Application.Resources>
</Application>
```